

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

PROGRAMACIÓN 3
Examen 1
(Primer semestre 2025)

Indicaciones generales:

- **Duración:** 3 horas.
- **Materiales o equipos a utilizar:**
La sección teórica se realizará sin material de apoyo.
En la sección práctica podrá utilizar material de apoyo: diapositivas, ejemplos de clase, grabaciones de clase, código fuente y documentación oficial de Microsoft u Oracle.
- No está permitido el uso de ningún material o equipo electrónico adicional al indicado.
- **La presentación, la ortografía y la gramática influirán en la calificación.**

Puntaje total: 20 puntos

PARTE TEÓRICA: (4 puntos)

Preguntas Teóricas

Responda en PAIDEIA las dos preguntas teóricas.

PARTE PRÁCTICA: (16 puntos)

Pregunta 1 (04 puntos) – **Obligatoria (JAVA – con Netbeans 25 – JDK 21)**

El patrón **Strategy** es un patrón de diseño que permite definir una familia de algoritmos, encapsular cada uno de ellos en clases separadas, y hacerlos intercambiables en tiempo de ejecución. El objetivo es permitir que el algoritmo varíe independientemente de los clientes que lo usan.

En lugar de implementar múltiples versiones de un algoritmo usando estructuras condicionales como **if** o **switch**, el patrón **Strategy** promueve la separación de responsabilidades, al delegar la lógica específica a clases concretas que implementan una misma interfaz común. Esto mejora la extensibilidad del software y facilita su mantenimiento.

En Java, el patrón **Strategy** se puede implementar de la siguiente manera:

1. Definir una interfaz común que declare el método que varía entre algoritmos.

```
public interface Estrategia {  
    void ejecutar();  
}
```

2. Crear clases concretas que implementen esa interfaz con comportamientos diferentes.

```
public class EstrategiaA implements Estrategia {  
    public void ejecutar() {  
        System.out.println("Ejecutando Estrategia A");  
    }  
}  
  
public class EstrategiaB implements Estrategia {  
    public void ejecutar() {  
        System.out.println("Ejecutando Estrategia B");  
    }  
}
```

3. Diseñar una clase contexto, que utilice una referencia a la estrategia y delegue en ella la operación que varía.

```
public class Contexto {  
    private Estrategia estrategia;  
  
    public Contexto(Estrategia estrategia) {  
        this.estrategia = estrategia;  
    }  
  
    public void ejecutarEstrategia() {  
        estrategia.ejecutar();  
    }  
  
    public void setEstrategia(Estrategia estrategia) {  
        this.estrategia = estrategia;  
    }  
}
```

4. Usar el patrón desde la lógica del cliente, eligiendo dinámicamente la estrategia deseada.

```
Contexto contexto = new Contexto(new EstrategiaA());  
contexto.ejecutarEstrategia();  
  
contexto.setEstrategia(new EstrategiaB());  
contexto.ejecutarEstrategia();
```

Se desea implementar el patrón **Strategy** para el siguiente problema: calcular los puntos acumulados de los conductores en función de las infracciones registradas y de ciertas reglas específicas.

Se deben definir tres estrategias distintas para el cálculo de puntos:

- **Estrategia 1:** Calcula los puntos acumulados sin considerar la fecha de adquisición del vehículo. Es decir, toda infracción registrada al vehículo sumará puntos al conductor, sin filtrar por fecha.
- **Estrategia 2:** Calcula los puntos sumando solo las infracciones registradas posteriormente a la fecha de adquisición del vehículo por parte del conductor. Es decir, solo se toman en cuenta las infracciones cometidas después que el conductor adquirió el vehículo.
- **Estrategia 3:** Aplica un descuento del 50% sobre los puntos calculados (como en la Estrategia 2). Se asumen en esta estrategia que el conductor ha pagado la multa con anterioridad.

Restricciones:

- La aplicación no debe conectarse a la base de datos; los datos necesarios se cargarán manualmente en las clases correspondientes. Si lo prefiere puede usar el **framework Junit**.
- Se debe usar el modelo de dominio basado en las entidades Conductor, Vehiculo, VehiculosPorConductor, Infraccion, y RegistroInfracciones (las estructuras y relaciones fueron descritas previamente en el caso).
- Se debe diseñar la interfaz común de la estrategia y las tres implementaciones concretas.
- Además, diseñe una clase contexto que permita seleccionar la estrategia en tiempo de ejecución y ejecutar el cálculo correspondiente.

Puede utilizar el archivo .java para instanciar objetos con datos.

A continuación, elija únicamente entre desarrollar la Pregunta 2 o la Pregunta 3.

No está permitido responder ambas.

En caso de hacerlo, se seleccionará una de las respuestas al azar para su calificación.

Pregunta 2 (06 puntos) – (JAVA – con Netbeans 25 – JDK 21)

En la Pregunta 1 se implementó el patrón **Strategy** para calcular los puntos acumulados por los conductores, considerando las infracciones registradas y reglas específicas. En esta pregunta, se tomará como base dicho enfoque, pero se desarrollarán nuevas estrategias y se integrará el acceso a datos mediante el patrón DAO para alimentar la lógica de cálculo de puntos.

Descripción del caso:

La Dirección General de Tránsito desarrolla un sistema para evaluar y seguir conductores con infracciones, gestionando puntos según la gravedad de las faltas. El sistema consulta datos de conductores y sus vehículos, calcula puntos según la política vigente de campañas activas, que pueden condonar puntos según la gravedad:

- Campaña "**Solo Muy Graves**": solo se cuentan infracciones muy graves; leves y graves se condonan.
- Campaña "**Graves y Muy Graves**": se cuentan graves y muy graves; leves se condonan.
- Modo General: todas las infracciones se consideran.

Para esto se debe separar la lógica de negocio y el acceso a datos.

Acceso a Datos:

En el módulo Maven **TransitSoftAccesoDatos**, implemente el método **obtenerPuntos(int idConductor, Gravedad gravedad)** dentro de la clase **ConductorDAOImpl**. Este método deberá retornar el total de puntos acumulados por un conductor, considerando todas las infracciones registradas en todos los vehículos asociados al conductor, filtrando exclusivamente aquellas cuya gravedad coincida con el parámetro recibido.

Para ello, deberá realizar una consulta a la base de datos que involucre las tablas **EX1_REGISTRO_INFRACCIONES** y **EX1_INFRACCIONES**, uniéndolas mediante el campo **INFRACCION_ID**, y filtrando por **CONDUCTOR_ID** y **GRAVEDAD**. El cálculo de puntos deberá basarse en la información contenida en estas tablas.

Puede implementar la consulta utilizando sentencias SQL o mediante procedimientos almacenados, según considere más adecuado.

Lógica de Negocio:

En el módulo Maven **TransitSoftNegocio**, implemente el método **calcularPuntos(int idConductor, TipoCampanha campanha)** en la clase **ConductorBOImpl**. Este método debe invocar la implementación adecuada en la clase **ConductorDAOImpl** según la campaña activa:

- Campaña "**Solo Muy Graves**": solo se cuentan las infracciones muy graves; leves y graves se condonan.
- Campaña "**Graves y Muy Graves**": se cuentan las infracciones graves y muy graves; leves se condonan.
- Modo General: se consideran todas las infracciones para el cálculo de puntos.

Para garantizar flexibilidad y escalabilidad, se debe aplicar el patrón **Strategy**, permitiendo calcular puntos según la campaña vigente sin modificar la lógica existente. Se solicita implementar las clases estrategias necesarias para poder hacer el cálculo de puntos adecuado de acuerdo con el tipo de campaña seleccionado.

Se le entrega esta clase Contexto que puede usar como base para su solución:

```
public class Contexto {
    private static final Map<TipoCampanha, EstrategiaCalculo> estrategias = Map.of(
        TipoCampanha.SOLO_MUY_GRAVES, new EstrategiaSoloMuyGraves(),
        TipoCampanha.GRAVES_Y_MUY_GRAVES, new EstrategiaGravesYMuyGraves(),
        TipoCampanha.MODO_GENERAL, new EstrategiaModoGeneral()
    );

    private EstrategiaCalculo estrategia;

    public Contexto(TipoCampanha campanha) {
        this.estrategia = estrategias.getOrDefault(campanha, new
EstrategiaModoGeneral());
    }

    public int ejecutar(int idConductor) {
        return estrategia.calcularPuntos(idConductor);
    }
}
```

Pregunta 3 (06 puntos) – (C# – con Visual Studio 2022 - .Net Framework 4.8.1)

Un **archivo binario** es un tipo de archivo que almacena datos en formato binario, es decir, en una secuencia de bits (0s y 1s) que no está pensada para ser leída directamente por las personas. Es por ello que, al abrirlas con un editor de texto, su contenido no resulta comprensible.

Se cuenta con un archivo binario llamado: "**reportes_infracciones.bin**", en el cual se han almacenado ciertos objetos de un determinado tipo, en este caso, específicamente, objetos de la clase "**ReporteInfraccion**".

Se solicita desarrollar un programa de consola en C# que, al ejecutarlo, realice lo siguiente:

1. Lea todos los objetos almacenados en el archivo binario mencionado y los cargue en memoria.
2. Realice la persistencia (registro) de cada uno de los objetos en la base de datos, específicamente en la tabla "**ex1_reportes_infracciones**". Puede utilizar el procedimiento almacenado "**INSERTAR_REGISTRO_INFRACCION**".

Utilice como base el proyecto que se encuentra en PAIDEIA, el cual contiene la clase "**ReporteInfraccion**" que fue utilizada para crear el archivo binario.

El algoritmo que permite leer los objetos en memoria desde el archivo binario debe colocarlo en la capa de negocio o Business.

Asimismo, se le brinda como referencia el siguiente código que permite leer objetos desde un archivo binario:

```
BindingList<Clase> lista = new BindingList<Clase>();

FileStream fs = new FileStream("reportes_infracciones.bin", FileMode.Open);
BinaryFormatter formatter = new BinaryFormatter();

try
{
    while (fs.Position < fs.Length)
    {
        Clase objeto = (Clase) formatter.Deserialize(fs);
        lista.Add(objeto);
    }
}
catch (Exception ex)
{
    Console.WriteLine("Error al deserializar: " + ex.Message);
}
finally
{
    fs.Close();
}
```

- Considere buenas prácticas de programación, optimización (evitar repetir código fuente), programación por capas (según lo aprendido en clase) y lectura de contraseña encriptada para la conexión a la base de datos.

Pregunta 4 (06 puntos) – **Obligatoria (C# – con Visual Studio 2022 - .Net Framework 4.8.1)**

En esta pregunta, se debe completar la funcionalidad del formulario de **registro de infracciones** de tránsito, permitiendo registrar nuevas infracciones para conductores y sus vehículos asociados.

Descripción de la funcionalidad:

- **Búsqueda de Conductor:** Al hacer clic en el botón "Buscar", se debe localizar al conductor en la base de datos mediante su número de licencia, mostrar su nombre completo en el campo "Conductor", cargar en la lista desplegable "Vehículo" todos los vehículos asociados y presentar los puntos acumulados en la parte inferior.
- **Selección de Infracción:** Al hacer clic en "Seleccionar", se abrirá un modal con el listado de infracciones. Una vez elegido la infracción correspondiente, la información se debe mostrar en la sección "Detalles de Infracción" y el modal se cerrará automáticamente.
- **Registro de infracción:** Al hacer clic en "Registrar Infracción", se debe validar que todos los campos requeridos estén completos (conductor, vehículo seleccionado, infracción seleccionada y fecha de infracción). Hay que asegurar que la fecha no sea futura. Luego, proceder a registrar la infracción en la base de datos junto con los puntos acumulados del conductor. Finalmente, mostrar un mensaje de éxito o error según corresponda.

Fig. 01. Formulario solicitado.

Recursos Provistos

Se proporciona una solución que incluye:

1. **Capa de Presentación:**
 - Proyecto ASP.NET WebForms pre-construido con la interfaz de usuario completamente implementada. **Deberá completar la funcionalidad solicitada.**
2. **Capa de Negocio:**
 - Proyecto con código fuente que contiene la lógica de negocio
 - **El alumno debe trabajar directamente con esta capa sin alterarla.**
3. **DLLs de Referencia:**
 - [InfraccionesDominio.dll](#): Contiene las clases del modelo de dominio.
 - [InfraccionesPersistencia.dll](#): Implementa el acceso a datos (DAO).

Nota: Antes de ejecutar la aplicación, debe configurar las credenciales de conexión a la base de datos en el archivo **Web.config** (no es necesario el password encriptado para esta pregunta).

```
<databaseSettings>
  <add key="host" value="[TU-SERVIDOR]"/>
  <add key="database" value="[TU-BASE-DE-DATOS]"/>
  <add key="user" value="[TU-USUARIO]"/>
  <add key="password" value="[TU-PASSWORD]"/>
  <!-- ...otros settings... -->
</databaseSettings>
```

Profesores del curso:

Dr. Freddy Paz
Dr. Andrés Melgar
Dr. Heider Sánchez
Dr. Eric Huiza

San Miguel, 23 de mayo de 2025